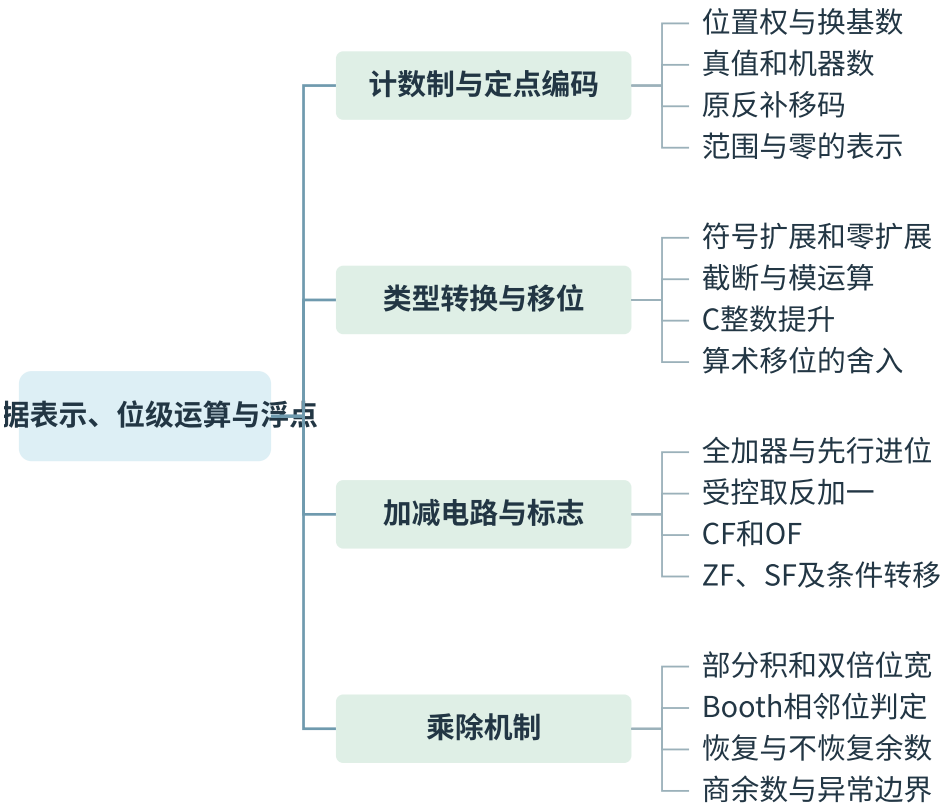
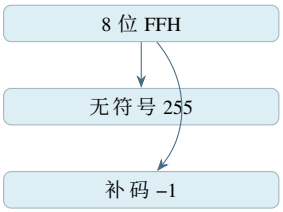
2 数据表示、位级运算与浮点数

知识导图



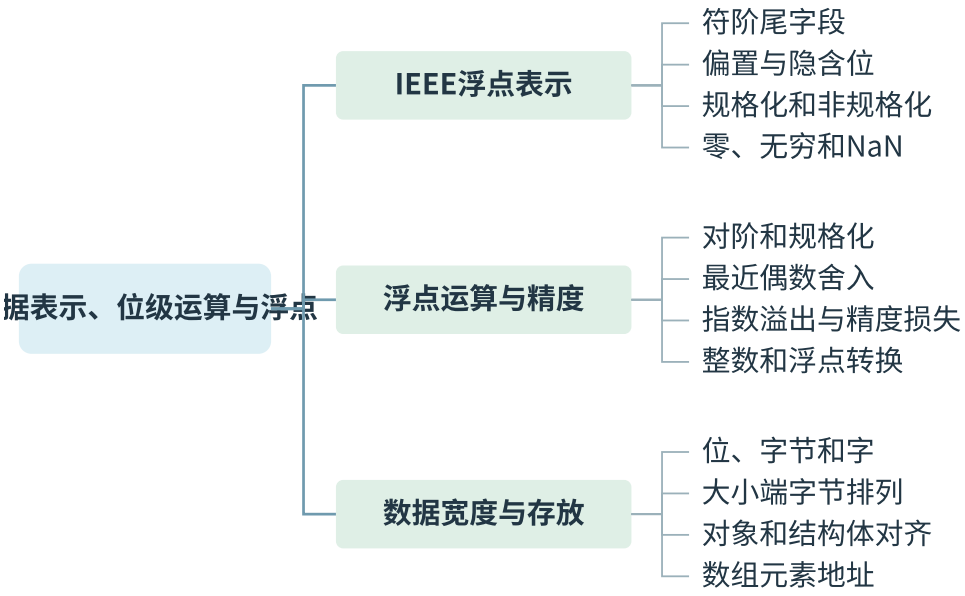
最小反例·最高位 1 并不总是负数

最小反例：同一 8 位串 FFH 按无符号解释是 255，按补码解释是 −1。编码和位宽必须与位串一同给出。直接把最高位称作负号，会把无符号扩展、浮点阶码和地址都误读成有符号数。



同一位串依编码得到不同真值。

知识导图（续）



不变量·范围来自位权

补码最高位权为负，其余位权为正；最小值只有最高位 1，最大值只有最高位 0。每次推范围先写位权，就能同时解释零唯一、负数多一个，以及最小负数同宽取负不可表示。

本章围绕 “计数制与定点编码 → 类型转换与移位 → 加减电路与标志 → 乘除机制 → IEEE 浮点表示 → 浮点运算与精度 → 数据宽度与存放” 展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

错项排除·扩展看源数解释

FFFAH 若是 unsigned short 则为 65530，扩宽补 0；若是 signed short 补码则为 −6，扩宽复制符号。目标名字中出现 unsigned 不代表先补 0：数值转换还需按目标模数处理。

2.1 位串与数值之间还差一个解释规则

定义 2.1.1 (概念与适用对象)

位是取 0 或 1 的二进制位，字节 B 在本讲义中为 8 位，bit 写作 b。同一位串可以解释成无符号整数、补码整数、浮点数或字符编码。计算前必须同时确定**位宽、编码和小数点位置**，不能看见最高位为 1 就断言 “这个数是负数”。

r 进制采用位置权，$(a_k \cdots a_0.a_{-1} \cdots)_r$ 的值是 $\sum_i a_i r^i$，其中 $0 \le a_i < r$。二进制与八进制、十六进制转换时，从小数点向两侧分组：每 3 位一组对应一位八进制，每 4 位一组对应一位十六进制；整数端左侧补 0，小数端右侧补 0。

2.1.1 例 2-1　整数与小数分别换基数

例 2.1.1 (配套例题)

【配套例题】若十进制数为 137.5，则其八进制数为（　）。

A. 89.8　B. 211.4　C. 211.5　D. 1011111.101

解析。$137 = 2 \times 8^2 + 1 \times 8 + 1$，小数部分 $0.5 \times 8 = 4$，因此为 $(211.4)_8$，选 **B**。八进制数字只能是 0 至 7，A 本身就不是合法八进制数。分组也可验证：$137.5 = (10001001.1)_2 = (010\ 001\ 001.100)_2$。

十进制整数转二进制可反复除 2 取余、倒序排列；小数则反复乘 2 取整数、顺序排列。有限十进制小数未必能写成有限二进制小数，例如 0.1 的二进制表示无限循环。保存有限位时会发生近似，这正是后面浮点舍入出现的原因之一。

2.2 定点编码：范围先于运算规则

真值是按通常数学符号写出的数值；机器数是采用约定编码保存的位串。定点表示约定小数点位置固定，并不表示存储器真的另存了一个小数点。

对 n 位整数，常用编码如下。

编码	最高位和数值含义	表示范围
无符号	所有位均有正权	0 至 $2^n - 1$
原码	最高位为符号，其余为绝对值	$-(2^{n-1} - 1)$ 至 $2^{n-1} - 1$，两个零
反码	正数同原码，负数的数值位取反	与原码相同，两个零
补码	最高位权为 -2^{n-1}，其余为正权	-2^{n-1} 至 $2^{n-1} - 1$，一个零

8 位补码 `1111 1101` 的值为 $-128+64+32+16+8+4+1 = -3$。也可先把它作为无符号数 253，再因最高位为 1 减去 256，得到 −3。后一方法适合十六进制机器数：对 n 位补码，若无符号解释为 u，则最高位为 0 时取 u，为 1 时取 $u - 2^n$。

求一个负整数的补码时，可在给定位宽下把它的绝对值逐位取反再加 1；位宽不能省。8 位中 -128 编码为 `1000 0000`，它的数学相反数 128 不能用 8 位补码表示。“取反加 1” 得到的仍是同一位串，这说明运算按模 256 进行，并不表示已经得到可表示的正 128。

原码中的 `1000 0000` 表示 -0，补码中则表示 -128。反码的 -0 是 `1111 1111`。对于常见的偏置 2^{n-1} 的整数移码，有 $E = x + 2^{n-1}$，与对应补码相比只需翻转最高位；IEEE 754 阶码的偏置通常为 $2^{k-1} - 1$，不可套用 “翻转最高位” 这一特殊关系。

若采用一位符号、$n-1$ 位小数的补码定点小数，最高位权为 -1，其余权从 2^{-1} 至 $2^{-(n-1)}$，范围为 $[-1, 1 - 2^{-(n-1)}]$。同样的位串改变小数点位置，真值就按 2 的幂缩放。

最小反例·右移不等于向零除法

最小反例：8 位补码 -3 为 FDH，算术右移得到 FEH，即 -2；C 整数 $-3/2$ 向零截断为 -1。两者相等只在额外条件下成立。检查奇数负数能快速暴露把右移无条件替代除法的错误。

2.3 截断、扩展与 C 类型转换

位宽变小时，硬件截取低位；能否保留原数值，要检查被丢掉的位。位宽增大时，无符号数在高位补 0，有符号补码数复制符号位。原码、反码各有自己的扩展规则，补码规则并不适用于所有编码。

8 位补码 -3 扩为 16 位是 `1111 1111 1111 1101`，无符号 253 扩为 16 位则是 `0000 0000 1111 1101`。扩展后低 8 位相同，真值不同。**扩展方式首先取决于源数怎样解释，而不是目标类型名字中有没有 unsigned**。

2.3.1 例 2-2　无符号短整数扩展

例 2.3.1 (配套例题)

【配套例题】假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

```
unsigned short x = 65530;
unsigned int y = x;
```

得到 y 的机器数为（　）。

A. 0000 7FFAH　B. 0000 FFFAH　C. FFFF 7FFAH　D. FFFF FFFAH

解析。$65530 = 65536 - 6 =$ `FFFAH`，源类型是无符号 16 位数，扩展补 0，得到 `0000 FFFAH`，选 **B**。最高位为 1 在这里是普通数值位，不是负号。

2.3.2 例 2-3　负的 short 转成 unsigned int

例 2.3.2 (配套例题)

【配套例题】在 32 位计算机上执行下列 C 语言代码段后，ui 的值是（　）。

```
short si = -32767;
unsigned int ui = si;
```

A. $2^{15} - 1$　B. $2^{15} + 1$　C. $2^{32} - 2^{15} - 1$　D. $2^{32} - 2^{15} + 1$

解析。按题目的 16 位 short、32 位 unsigned int 模型，-32767 的 16 位补码为 `8001H`。数值转换为 32 位无符号整数后，得到与 -32767 模 2^{32} 同余的数，即 $2^{32}-32767=2^{32}-2^{15}+1$，机器数为 `FFFF 8001H`，选 **D**。把 `8001H` 直接补 0 会把负的源值误当成无符号 32769。

C 中的 usual arithmetic conversions 也会改变比较方式。若 32 位 `int i=-1` 与 32 位 `unsigned u=1` 比较，常见模型下 i 转为无符号 $2^{32}-1$，所以 `i<u` 为假。需要先完成整数提升和通常算术转换，再决定按哪种值比较。

本讲义中 “截断取低位” 的位级推演按题设机器模型进行。C 语言本身不保证有符号溢出按补码绕回：有符号加法溢出属于未定义行为；按本讲义采用的 C11/C17 口径，超范围转换到有符号类型还涉及实现规定。考题若明确给机器位宽和截断模型，可据模型计算；编写可移植 C 程序时不能把这些题设当成语言的普遍保证。

2.3.3 移位先问逻辑还是算术

逻辑右移高位补 0，算术右移复制符号位；左移均在低位补 0，但如何解释结果及是否溢出仍取决于编码。8 位补码 -3 算术右移 1 位得到 -2，相当于向负无穷方向取整；整数除法 $-3/2$ 通常向零截断得到 -1，两者不能无条件互换。左移时被丢弃的高位无法恢复，“低位补 0” 也不保证数值恰好乘 2。

2.4 加法器怎样同时支持加与减

一位全加器接收 A_i, B_i, C_i，产生和位 S_i 与向高位的进位 C_{i+1}：

$$S_i = A_i \oplus B_i \oplus C_i, \qquad C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i.$$

定义进位生成 $G_i = A_iB_i$、进位传递 $P_i = A_i \oplus B_i$，即可逐层展开进位表达式。串行进位需等低位进位逐级传播；先行进位用更复杂的组合逻辑并行形成若干进位，缩短关键路径。更快的进位结构没有改变加法的数值规则。

加减运算器用控制位 Sub 将第二输入按位与 Sub 异或，同时令最低进位 $C_0 = \text{Sub}$。Sub=0 时执行 $A+B$；Sub=1 时执行 $A+\overline{B}+1$，即模 2^n 的减法。算术逻辑部件 ALU 还通过选择控制实现与、或、异或等逻辑操作；移位器是否算在 ALU 内部，要看具体的数据通路图。

2.4.1 例 2-4　减法器的两路输入不直接送负数符号

例 2.4.1 (配套例题)

【配套例题】某计算机字长为 8 位，CPU 中有一个 8 位加法器。已知无符号数 $x=69, y=38$，若在该加法器中计算 $x-y$，则加法器的两个输入端信息和输入的低位进位信息分别为（　）。

A. 0100 0101、0010 0110、0
B. 0100 0101、1101 1001、1
C. 0100 0101、1101 1010、0
D. 0100 0101、1101 1010、1

解析。$69 = 0100\,0101$，$38 = 0010\,0110$。第二路取反为 `1101 1001`，另由最低进位补上 1，所以选 **B**。若第二路已取反加 1 又令最低进位为

不变量·减法中的一只加一

控制 Sub=1 时第二路逐位取反，最低进位再加 1，总体才是 A−B。若已经送入 B 的负数补码，又把最低进位设 1，就把那一个 1 重复加入。追踪输入端比只写 “取补码” 更精确。

1，就重复加了 1。运算实际为 $69+217+1=287$，低 8 位 31、最高进位 1，符合 $69-38=31$。

错项排除·进位和有符号溢出分开

CF 依据无符号加减的进借位，OF 依据补码真值范围。同一运算完全可以 CF=1 而 OF=0，或反过来。先确定 ISA 对减法标志采用借位还是无借位口径，再套公式。

2.5 CF 与 OF 分别判断什么

同一次加法器运算可以同时按有符号和无符号两种规则解释，因此产生不同用途的标志。ZF 表示结果低 n 位是否全 0，SF 等于结果最高位；SF=1 并不自动表示数学真值为负，还要确认按有符号解释且未发生有符号溢出。

对于加法, CF 反映最高位向外进位; 对于本讲义采用的 “CF=1 表示借位” 约定, 减法的 CF 为最高进位的反值。可统一写为 $\mathrm{CF}=C_n\oplus\mathrm{Sub}$。某些 ISA 用 “无借位” 作为减法进位标志，遇到这类题设时，必须按该 ISA 的约定判断。

OF 判断有符号结果是否越出补码范围：加法时同号相加而结果异号会溢出；减法时两操作数异号且结果符号与被减数不同会溢出。也可用符号位的进位输入与进位输出异或：$\mathrm{OF}=C_{n-1}\oplus C_n$。它不要求把符号位当成独立的符号去做原码运算。

2.5.1 同一个结果，按哪一种数判断

位串运算、数值解释和条件判断应分三步。先按位宽完成加减，保留低 n 位；再按无符号或补码规则解读；最后选择与这种解释对应的标志。不能为了得到希望的正负号，改写已经产生的机器结果。

8 位运算	保留结果	按无符号解释	按补码解释
`7FH+01H`	`80H`	$127+1=128$，可表示	$127+1$ 超过 127
`FFH+01H`	`00H`	$255+1$ 超过 255	$-1+1=0$，可表示
`80H-01H`	`7FH`	$128-1=127$，无借位	$-128-1$ 小于 -128
`00H-01H`	`FFH`	$0-1$ 需借位	$0-1=-1$，可表示

同一过程的其余状态（按首列对应）：

8 位运算	CF、OF
`7FH+01H`	0、1
`FFH+01H`	1、0
`80H-01H`	0、1
`00H-01H`	1、0

ZF 和 SF 只看保留下来的结果：上表第二行 ZF=1，第一行与第四行 SF=1。若要由减法结果判断补码数的有符号小于或大于关系，应结合 OF 修正结果符号；例如第三行 SF=0，却不能据此说原来的 -128 大于 1。是否相等只需看 ZF，不必额外使用 OF。编码范围决定 “能否表示”，运算电路决定 “产生哪些位”，转移条件决定 “怎样使用标志”，三者要连起来判断。

2.5.2 例 2-5　十六进制补码相减

例 2.5.1 (配套例题)

【配套例题】假定有符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值及 x−y 的机器数分别是（　）。

A. x=−65，y=41，x−y 的机器数溢出
B. x=−33，y=65，x−y 的机器数为 FFFF FF9DH
C. x=−33，y=65，x−y 的机器数为 FFFF FF9EH
D. x=−65，y=41，x−y 的机器数为 FFFF FF96H

解析。`FFFFFFDFH` 距 2^{32} 差 33，故 x=−33；`41H` 是十进制 65。数学结果为 -98，在 32 位补码范围内，机器数为 $2^{32}-98=$ `FFFFFF9EH`，选 **C**。十六进制的 41 不能按十进制 41 读。

课程连接·标志接到条件分支

指令系统用 ZF 判断相等，用 CF 处理无符号大小，用 SF 与 OF 的组合处理有符号大小。编译同一个小于号时还要看操作数类型；电路相同，不代表转移条件可以混用。

2.5.3 例 2-6　有符号减法为何仍会产生借位

例 2.5.2 (配套例题)

【配套例题】已知 x、y 为 int 类型，当 x=100、y=200 时，执行“x 减 y”指令得到的溢出标志 OF 和借位标志 CF 分别为 0、1。那么当 x=10、y=−20 时，执行该指令得到的 OF 和 CF 分别是（　）。

A. OF=0, CF=0　B. OF=0, CF=1　C. OF=1, CF=0　D. OF=1, CF=1

解析。有符号真值为 $10-(-20)=30$，可表示，OF=0。若按无符号解释输入位串，-20 对应 $2^{32}-20$，10 小于它，减法需要借位，CF=1，选 **B**。题目已经用 100−200 说明了借位标志约定，不能把两个标志都按有符号真值判断。

条件转移利用这些标志完成比较。先计算 $a-b$：无符号 $a<b$ 对应 CF=1，无符号 $a>b$ 对应 CF=0 且 ZF=0；带符号 $a<b$ 对应 SF $\neq$ OF，带符号 $a>b$ 对应 ZF=0 且 SF=OF。用 SF 单独判断带符号大小，会在溢出时失败。

2.6 乘除运算必须保留的中间量

2.6.1 无符号乘法：为什么需要部分积和双倍位宽

两个 n 位无符号数的精确乘积最多需要 $2n$ 位。以王道书 §2.2 中的 13 乘 11 为例：$13=1101_2$，$11=1011_2$。乘数从低位到高位为 1、1、0、1，部分积依次是 13、26、0、104，总和 143，即 `1000 1111`。

顺序乘法器可以每轮检查乘数最低位，决定是否把被乘数加到部分积高半部，再将“进位—部分积—乘数”组合右移。部分积开始为 0，循环 n 次后得到 $2n$ 位积。相应的等价算法也可让被乘数左移、乘数右移，把选中的部分积加到双倍宽累加器；两种寄存器布局的逐轮数值不同，不应把一种布局的移位规则套用到另一种。

对 13×11，用双倍宽累加器的轨迹如下：

本轮乘数低位	本轮被乘数	累加器更新后	乘数右移后
1	13	13	5
1	26	39	2
0	52	39	1
1	104	143	0

带符号乘法不能把负数的补码直接当成同宽无符号数求完整积。Booth 法在乘数末端附一位 0，考察相邻两位 q_iq_{i-1}：01 时加被乘数，10 时减被乘数，00 或 11 不加减，再对组合寄存器算术右移。这里的前提

是符号扩展到位、中间寄存器足够宽；对最小负数取负时，尤其需要保护位。

以 4 位补码的 -3×5 说明 Booth 法（原创机制示例）。乘数寄存器 Q 为 4 位，附加位 q_{-1} 为 1 位，A 采用 5 位补码以保留保护位，被乘数 M=-3 也扩成 5 位。初态 A=`00000`、Q=`0101`、$q_{-1}=0$。每轮先看 Q_0q_{-1}，对 A 加减 M，再把 A、Q、q_{-1} 连接成 10 位整体算术右移；A 的最低位移入 Q 最高位，Q 最低位移入 q_{-1}。

轮次	判断位	移位前操作及 A 真值	移位后的 A
1	10	A−M=3	00001
2	01	A+M=−2	11111
3	10	A−M=2	00001
4	01	A+M=−2	11111

同一过程的其余状态（按首列对应）：

轮次	移位后的 Q	q_{-1}
1	1010	1
2	0101	0
3	0010	1
4	0001	0

四轮后 AQ 为 9 位 `11111 0001`，去掉冗余符号保护位即 8 位 `1111 0001`，真值为 -15。移位时必须复制 A 的符号位，并在寄存器边界传位；把 A、Q 分别右移会丢掉跨界位，得到错误乘积。

阵列乘法器用组合电路完成部分积生成与压缩求和，可能在一个足够长的时钟周期内完成；顺序乘法器复用 ALU 和移位器，通常跨多个周期。“一个周期”不能脱离所用电路和时钟周期长度讨论。常数乘法可以在保持规定语义的前提下化为移位与加减，例如 $13x=(x\ll3)+(x\ll2)+x$。

不变量·余数必须最终回验

无符号除法结束后，同时核 N=QD+R 及 $0\le R<D$。恢复法与不恢复法可以出现不同中间余数，但最终条件相同；不恢复法若结束时 R 为负，常需补做一次加 D 校正。

2.6.2 除法：商来自每一步余数比较

以无符号 7÷3 为例，按被除数的有效位 `111` 从高到低处理，初始余数 R=0。每读入一位，先令 $R\leftarrow 2R+b$，若 R≥3 则减 3 并在商中记 1，否则记 0。

读入位	比较前余数	本位商	本步余数
1	1	0	1
1	3	1	0
1	1	0	1

商为 `010` 即 2，余数 1，满足 $7=2\times3+1$ 且 $0\le R<3$。硬件通常固定处理整个位宽，在前面补零位，不会因此改变结果。恢复余数法先试减，结果为负时再加回除数；不恢复余数法根据余数符号决定下一轮加还是减，用不同轨迹完成同一任务。

不恢复余数法也可用同一 7÷3 观察。令余数 R 为 4 位有符号寄存器、Q 为 3 位，除数 D=3，初态 R=0、Q=`111`。每轮先把 R、Q 连接左移一位，Q 最高位移入 R 最低位；若上一轮 R 非负，本轮减 D，若上一轮 R 为负则加 D；运算后 R 非负就在 Q 最低位置 1，否则置 0。

轮次	左移后 R 真值	加减操作	操作后 R
1	1	R−D	−2
2	−3	R+D	0
3	1	R−D	−2

同一过程的其余状态（按首列对应）：

轮次	记商后的 Q
1	110
2	101
3	010

最后 R 仍负，补做 R←R+D，得到余数 1，Q=`010` 为商 2。第二轮没有先把 −2 恢复成 1 再试减，而把恢复效果合并进下一轮加法，这正是不恢复余数法的含义；最终余数校正不能漏掉。硬件布局或商编码约定不同会改变表中轨迹，做题时应遵循同一套规则。

有符号除法还要确定商、余数符号和舍入方向。按 C 向零截断模型，−7÷3 的商为 −2、余数为 −1，仍满足被除数 = 商 × 除数 + 余数。除数为 0 时需要异常处理；$-2^{n-1}/(-1)$ 的商超出 n 位补码范围。某种除法器可能检测到被除数绝对值小于除数，就提前输出商 0；另一种固定轮数除法器仍运行完整迭代。只有题干电路给出提前退出逻辑时，才能比较哪组数据用时最短。

错项排除·字段长度不等于精度

binary32 尾数字段 23 位，正规格化数隐含前导 1，因此有效精度 24 位；非规格化数隐含 0。把所有编码都当 1.F，会把最小正数、零和特殊值混在一起。先判断阶码是否全 0 或全 1。

2.7 IEEE 754：字段既规定精度，也规定特殊情况

IEEE 754 binary32 即通常的单精度格式，由 1 位符号 s、8 位阶码 E、23 位尾数字段 F 构成；binary64 分别为 1、11、52 位。这里 F 的整数值要除以 2^{23} 或 2^{52} 才是小数尾数部分。

项目	binary32（单精度）	binary64（双精度）
总位数	32 位	64 位
符号位 s	1 位	1 位
阶码 E	8 位	11 位
尾数字段 F	23 位	52 位
阶码偏置 B	127	1023
规格化数的真阶码 e	$-126 \le e \le 127$	$-1022 \le e \le 1023$
规格化数的有效精度	24 位，含隐含前导 1	53 位，含隐含前导 1
最小正规格化数	2^{-126}	2^{-1022}
最小正非规格化数	2^{-149}	2^{-1074}
最大有限正数	$(2-2^{-23})2^{127}$	$(2-2^{-52})2^{1023}$

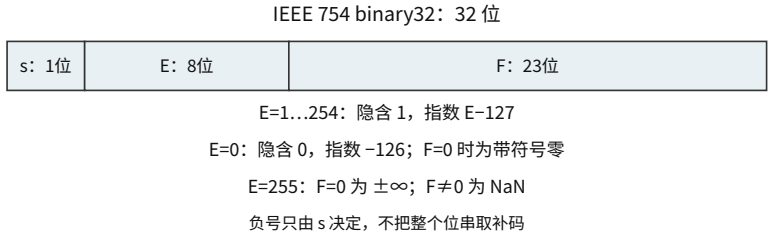


Figure 2.1: 单精度字段及三类指数编码

解码时先看阶码是否全 0 或全 1，再看尾数字段。下表中 p 是尾数

字段位数，binary32 取 23、binary64 取 52；B 是对应的阶码偏置，F 按无符号整数解释。

类别	阶码 E	尾数字段 F	数值或含义
正零、负零	全 0	0	$(-1)^s 0$，符号位区分正负零
非规格化数	全 0	非 0	$(-1)^s (F/2^p) 2^{1-B}$，隐含前导 0
规格化数	既非全 0，也非全 1	任意	$(-1)^s (1+F/2^p) 2^{E-B}$，隐含前导 1
正无穷、负无穷	全 1	0	$(-1)^s \infty$
NaN	全 1	非 0	非数；不能当作无穷或普通实数

不变量·阶码字段先减偏置

普通 IEEE 浮点数的指数是 E 减偏置，尾数字段另按二进制小数解释。整数移码常见的翻转符号位关系，不能直接套到偏置 127 的 IEEE 阶码。编码和解码各走一遍可验证字段。

对于 binary32：

$$x = \begin{cases} (-1)^s (1 + F/2^{23}) 2^{E-127}, & 1 \le E \le 254; \\ (-1)^s (F/2^{23}) 2^{-126}, & E = 0; \\ (-1)^s \infty, & E = 255, F = 0; \\ \text{NaN}, & E = 255, F \neq 0. \end{cases}$$

规格化数有隐含的前导 1，有效精度 24 位。非规格化数前导为 0，最小指数同样取 -126，数值逐渐接近 0；当 F=0 时是带符号零。非规格化数不能再用指数 E−127 配隐含 1 计算。

最小正规格化数为 2^{-126}，最小正非规格化数为 2^{-149}，最大有限正数为 $(2-2^{-23})2^{127}$。这些是数值范围；“能连续精确表示的整数” 是另一问题。binary32 能精确表示 $[-2^{24}, 2^{24}]$ 内所有整数，更大的某些整数仍能精确表示，但相邻可表示数的间隔可能大于 1。

2.7.1 例 2-7　负小数怎样组成单精度机器数

例 2.7.1 (配套例题)

【配套例题】−0.4375 的 IEEE 754 单精度浮点数表示为（　）。

A. BEE0 0000H　B. BF60 0000H　C. BF70 0000H　D. C0E0 0000H

解析。$0.4375 = 0.0111_2 = 1.11_2 \times 2^{-2}$。

s=1，E=−2+127=125=`01111101`，F 为 `110` 后补 20 个 0。

连接为 `1|01111101|11000000000000000000000`，十六进制为 `BEE00000H`，选 **A**。负数只改变符号位，不对浮点数全部位取补码。

2.7.2 例 2-8　同为负浮点数时怎样比较

例 2.7.2 (配套例题)

【配套例题】float 型数据常用 IEEE 754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f1 和 f2 中，若 `(f1)=CC90 0000H`、`(f2)=B0C0 0000H`，则 x 和 y 之间的关系为（　）。

A. x<y 且符号相同　B. x<y 且符号不同　C. x>y 且符号相同　D. x>y 且符号不同

解析。两个最高位都为 1，均为负。x 的阶码为 153，实际指数 26；y 的阶码为 97，实际指数 −30。两者均为有限规格化数，x 的绝对值远大于 y，因此 x<y，选 **A**。此处不必把尾数展开为完整十进制数。对负数，绝对值更大的反而更小。

int 转 float 可能丢低位，32 位 int 转 binary64 则可精确表示，因为 binary64 有 53 位有效精度。float 转 double 在通常 IEEE 格式下能精确保留原来的有限数值，但不能恢复 float 形成时已经丢掉的信息。浮点转整数按语言规则截断，且须检查范围；NaN 与任何数（包括自身）的相等比较都为假，不能把 `x/x==1` 用在 x=0 或无穷等情形。

最小反例·不溢出仍可不精确

最小反例：binary32 能精确存 2 的 24 次方，却不能精确存它加 1。指数远未到上限，有效位已经不足。浮点溢出与舍入损失要分别检查，答案有限并不保证与数学精确值相等。

2.8 浮点加减：对阶丢位以后还要判断舍入

两个浮点数只有指数相同，才能直接把有效数相加。通常先把较小指数的有效数右移，使指数与较大者一致；这叫对阶。大阶向小阶对齐会要求更多高位和更大的寄存器，不符合固定精度运算的通常实现。

对阶之后进行尾数运算，再规格化和舍入。加法可能得到 $10.xxx_2$，须右移 1 位、阶码加 1；异号相加发生抵消，可能得到 $0.00\cdots1xxx_2$，须左移、阶码减少。舍入使尾数加 1 时还可能再次进位，必须再检查规格化和阶码范围。若得到精确的 0，也不能继续无止境地左移寻找前导 1。

用一个仅保留 4 位有效数的教学格式说明舍入。$x = 1.000_2 \times 2^0$，$y = 1.000_2 \times 2^{-4}$，精确和为 1.0001_2，恰好位于保留为 `1.000` 和 `1.001` 的中点。采用 “舍入到最近，正中取偶数”，保留最低位为 0 的 `1.000`。若 x 改为 `1.001`，精确和为 `1.0011`，则正中取偶数进为 `1.010`。偶数指保留有效数的最低位为 0，不是判断整个真值是奇整数还是偶整数。

实际实现常保留保护位、舍入位和粘滞位。粘滞位记录更低位中是否出现过 1，因此 `1000`…与 `1000`…`1` 的舍入结论可能不同，不能只保留移出的第一位。

2.8.1 例 2-9　哪些步骤可能造成指数越界

例 2.8.1 (配套例题)

【配套例题】下列有关浮点数加减运算的叙述中, 正确的是（　）。

I. 对阶操作不会引起阶码上溢或下溢

II. 右规和尾数舍入都可能引起阶码上溢

III. 左规时可能引起阶码下溢

IV. 尾数溢出时，结果不一定溢出

A. 仅 II、III　B. 仅 I、II、IV　C. 仅 I、III、IV　D. I、II、III、IV

解析。对阶把较小阶提高到较大阶，最终阶码仍是原有合法阶码之一；右移尾数可能损失精度，但不使该阶码越界，I 正确。右规和舍入后的再右规会使指数加 1，可能越过上界，II 正确。抵消后的左规要减小指数，可能进入非规格化区或触及题设的下溢界，III 正确。尾数暂时形成 `10.xxx` 还可以右规，只要指数容纳得下，结果仍可表示，IV 正确。选 **D**。尾数的暂时进位与最终浮点范围溢出是不同层次。

采用 IEEE 754 的渐进下溢时，指数过小并不立即把所有结果变成 0，还可能形成非规格化数。题目若采用 “阶码、尾数均为补码” 的教学格式，就应按其位宽、小数点位置、双符号位和舍入方法计算，不能把该格式的尾数补码接到 IEEE 字段中。

浮点加法不满足数学实数的结合律。例如在 binary32 中，$a = 2^{25}, b = -2^{25}, c = 1$：$(a+b)+c = 1$，而 $a+(b+c)$ 中 b+c 可能先舍入回 b，结果为 0。发生差异的原因是中间舍入，不是加法指令随意改变数学规则。

课程连接·编程中的循环边界

无符号 n=0 时 n−1 绕到最大值，会改变循环终止；有符号溢出又涉及语言规则。把循环、编码范围、浮点精度三步分开检查，可以说明同一算法在不同类型中为何产生不同结果。

2.8.2 定点整数与浮点数的范围、精度怎样比较

32 位补码整数把 31 位用于正数的数值范围，范围内整数连续、没有空缺；binary32 把位分给符号、指数和有效数，扩大数值范围，同时只保留 24 位有效二进制数字。浮点数在某个数量级的间隔随指数增大，因而 “32 位” 相同不表示能精确保存同一批整数。

例如 2^{24} 可被 binary32 精确表示，但 $2^{24}+1$ 正好落在相邻两个可表示数之间；按最近偶数舍入回 2^{24}。而 2^{25} 仍可精确表示。问 “所有不超过某界的整数都精确” 与问 “某个大整数是否精确”，必须分别作答。整数溢出检查编码范围；浮点溢出检查舍入、规格化后的指数；尚未溢出的浮点结果也可能已经不精确。

2.8.3 例 2-10　同一求和程序中的循环边界、整数范围与浮点精度

例 2.8.2 (配套例题)

【配套例题】已知 $f(n) = \sum_{i=0}^{n} 2^i = 2^{n+1} - 1 = \underbrace{11\cdots1}_{n+1\text{位}}{}_2$，计算 $f(n)$ 的 C 语言函数 f1 如下。

```c
int f1(unsigned n) {
    int sum = 1, power = 1;
    for (unsigned i = 0; i <= n - 1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

将 f1 中的 int 都改为 float，可得到计算 $f(n)$ 的另一个函数 f2。假设 unsigned 和 int 型数据都占 32 位，float 采用 IEEE 754 单精度标准。请回答下列问题。

（1）当 n=0 时，f1 会出现死循环，为什么？若将 f1 中的变量 i 和 n 都定义为 int 型，则 f1 是否还会出现死循环？为什么？

（2）f1(23) 和 f2(23) 的返回值是否相等？机器数各是什么（用十六进制表示）？

（3）f1(24) 和 f2(24) 的返回值分别为 33 554 431 和 33 554 432.0，为什么不相等？

（4）$f(31) = 2^{32} - 1$，而 f1(31) 的返回值却为 −1，为什么？若使 f1(n) 的返回值与 f(n) 相等，则最大的 n 是多少？

（5）f2(127) 的机器数为 7F80 0000H，对应的值是什么？若使 f2(n) 的结果不溢出，则最大的 n 是多少？若使 f2(n) 的结果精确（无舍入），则最大的 n 是多少？

解析。本题把同一数列放进不同表示中：先查循环是否结束，再查运算范围，最后查浮点有效位。以下对 f1(31) 的既定结果和 n=0 时持续循环的轨迹，都按 32 位补码机器执行模型解释；标准 C 中有符号溢出

属于未定义行为，n=0 时循环体也可能先发生这种溢出，不能把本题机器推演当成可移植 C 保证。

（1）i 和 n 均为 unsigned，n=0 时，`n-1` 按模 2^{32} 运算成为 $2^{32}-1$。任何 32 位无符号 i 都满足 `i<=n-1`，i 达到最大值后再加 1 又回到 0，因此循环条件一直为真。若两者都改为 int，$n-1$ 为 -1，第一次测试 $0 \le -1$ 就为假，循环一次也不执行，不会因这个边界形成死循环。

（2）$f(23)=2^{24}-1=16\,777\,215$，是 24 个连续的二进制 1。它在 32 位 int 范围内，且 binary32 恰有 24 位有效精度，故两者数值相等。f1(23) 的补码机器数为 `00FF FFFFH`。浮点形式为 $1.\underbrace{11\cdots1}_{23\text{位}}\times2^{23}$，s=0、E=23+127=150、F 全 1，故 f2(23) 的机器数为 `4B7F FFFFH`。相同数值不要求机器编码相同。

（3）$f(24)=2^{25}-1$ 需要 25 位有效数字，int 仍能精确保存，binary32 却只能保留 24 位。最后一次加法的精确结果为 33 554 431，处于 33 554 430 与 33 554 432 之间的正中；最近偶数舍入选后者，得到 2^{25}，机器数 `4C00 0000H`。这里尚未浮点溢出，只丢失了低位精度。

（4）$2^{32}-1$ 超过 32 位补码 int 最大值 $2^{31}-1$。按题设保留的低 32 位全为 1，解释为补码 -1。要求所有返回值与数学数列一致，须 $2^{n+1}-1 \le 2^{31}-1$，所以最大 n=30；不能因为无符号 32 位能表示 $2^{32}-1$，便认为 int 也能表示。

（5）`7F80 0000H` 的阶码全 1、尾数全 0、符号 0，表示正无穷。n=126 时，精确数 $2^{127}-1$ 被舍入为 2^{127}，仍是有限数；n=127 时，结果越过最大有限数并舍入为正无穷。因此不溢出的最大 n=126。要完全精确，数列中 n+1 个连续 1 必须全部容纳在 24 位有效数中，最大 n=23。两个上界分别来自指数范围与有效精度，不能互换。

2.9 大端、小端和对齐解决不同问题

多字节数据的大端方式把最高有效字节放在最低地址，小端方式把最低有效字节放在最低地址。字节序只改变一个多字节对象内部的字节排列，不反转每个字节内的位，也不改变数组元素的顺序。

32 位值 `12345678H` 从低地址到高地址，大端为 12、34、56、78，小端为 78、56、34、12。讨论“高位字节”时指数字意义上的高位，讨论“高地址”时指存储位置，两个维度应分开。

对齐指对象的起始地址满足规定的倍数要求。常用的自然对齐模型中，2 B 对象的起始地址是 2 的倍数，4 B 对象是 4 的倍数。编译器还可能为结构体在成员之间和末尾加入填充，使各成员地址和相邻数组元素都满足对齐规则。具体要求属于 ABI/编译器约定；题目给出“按边界对齐”时，才按给定类型大小计算。

不变量·字节序和对齐分两步

先依成员类型与对齐规则求对象布局，再把每个值拆成字节按大小端放入。字节序不改变结构体填充，也不反转数组元素次序。两步分别核对后，才能回答某地址保存的具体字节。

2.9.1 例 2-11　先确定结构布局，再看低字节

例 2.9.1 (配套例题)

【配套例题】某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct {
    int a;
    char b;
```

```
    short c;
} record;
record.a = 273;
```

若 record 变量的首地址为 0xC008，则地址 0xC008 中的内容及 record.c 的地址分别为（ ）。

A. 0x00，0xC00D　　B. 0x00，0xC00E
C. 0x11，0xC00D　　D. 0x11，0xC00E

解析。a 占偏移 0 至 3，b 占偏移 4。下一地址偏移 5 不是 2 的倍数，须填充 1 B，c 占偏移 6、7，故 c 地址为 C00EH。$273 = \texttt{00000111H}$，小端最低地址保存低字节 11H，选 **D**。填充来自对齐，低字节 11 来自字节序，两步不能互相代替。

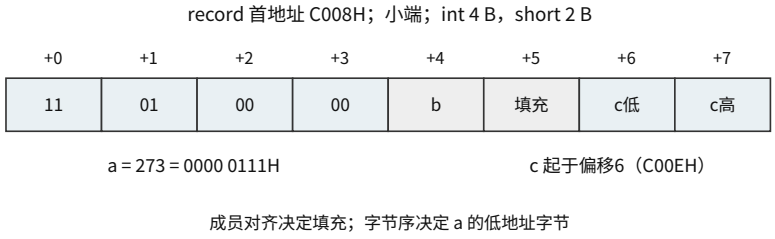


Figure 2.2: 结构体成员、填充与低地址字节分别标明

二维数组按行优先存放时，`a[i][j]` 的元素偏移为 $i \times$ 列数 $+ j$，字节偏移还要乘元素字节数。对于取负值的 short，先写出完整的 16 位补码，再按字节序拆开。例如 -6 为 `FFFAH`，小端低地址是 FA、下一字节是 FF，不是把真值“−6”拆成十进制字符。

2.10 把位串、真值和条件分开核算

表示与运算题的统一路径

先写位宽与编码，再求数学真值或完整精确结果；按题设完成截断、扩展或舍入，最后解释结果位串并检查标志。浮点解码先识别阶码的全 0/全 1，普通数公式不适用于特殊编码。涉及 C 代码时再核语言语义，不能把某种硬件结果提升为标准 C 保证。

2.10.1 补码为什么恰好多一个负数

由位权得到范围

n 位补码的值是 $-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$。最小时只有符号位为 1，值为 -2^{n-1}；最大时符号位 0 而其余全 1，值为 $\sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1$。这既说明负数多一个，也说明最小负数的相反数无法放回同宽补码中。

□

2.10.2 精度与范围各问什么

binary32 在 $[2^e, 2^{e+1})$ 内，相邻正规格化数相差 2^{e-23}。原因是尾数字段每增加 1，有效数增加 2^{-23}，再乘指数倍率 2^e。在 $[2^{24}, 2^{25})$ 间距为 2，

不变量·相邻间隔随指数变大

正规格化 binary32 在同一指数区间内间隔是 2 的 e−23 次方。指数每增 1，相邻间隔也翻倍。先算间隔再判某整数是否可表示，比只记 “24 位精度” 更容易处理大整数与舍入中点。

所以 $2^{24}+1$ 无法精确保存；2^{25} 本身却是精确的二进制幂。有限数是否溢出取决于指数边界，能否精确取决于有效位，两者不可互代。

最近偶数舍入先比较被舍弃部分与半个间隔：小于一半向较近值，大于一半向另一侧；恰好一半时选保留末位为 0 的候选。只写 “四舍五入” 会漏掉二进制等距点规则。

2.10.3 对齐和容量的单位

机器字长是 CPU 一次处理整数的基本宽度，存储字长是一个存储单元保存的位数，指令字长是某条指令的编码位数；三者可以不同。按字节编址时，32 位整数通常占 4 个连续地址；按 32 位字编址时，同一个字只占一个地址。先把地址单位写出来，再计算数组偏移、总容量和跨界访问次数。

自然对齐要求对象起点满足某个倍数，但具体倍数来自题设或 ABI。结构体末尾填充使数组中下一对象也满足对齐；删除末尾填充可能让第一个对象内部正确而第二个对象起点出错。字节序则只回答多字节值的高低有效字节如何落到地址，不能决定填充量。

课程连接·单位衔接存储系统

数据宽度、地址单位和块大小会在下一章共同决定偏移位数与容量。按字节编址时 4B 整数跨 4 个地址，但同一字节内 8 个位并不产生 8 个独立字节地址。每个对数前都先写清以字节还是以字计数。

2.11 主动回忆自测

2.11.1 配套自测

1. 原创补充：8 位机器计算 `7FH+01H`，按本章标志约定求低 8 位结果、CF、OF、SF、ZF。
2. 原创补充：binary32 位串 `0000 0001H` 与 `0080 0000H` 各表示什么正数？两者是否都具有隐含前导 1？
3. 【配套例题】已知 32 位 int 变量 i=12345，float 变量 $f=1.2345\times 2^3$。下列结果为假的是：A. `i==(int)(double)i`；B. `f==(float)(double)f`；C. `i==(int)(float)i`；D. `f==(float)(int)f`。

答案。1. 结果 `80H`，CF=0、OF=1、SF=1、ZF=0；127+1 越出补码范围，但未越出 8 位无符号范围。2. 分别为 2^{-149} 与 2^{-126}，前者非规格化、隐含 0，后者规格化、隐含 1。3. D。12345 能被 binary32 精确表示；float 到 double 再回来也保留原 float 值，但 f 截断为整数会丢小数部分。

2.11.2 遮住答案复述导图

主动回忆·自拟

4. 配套结构体案例：某 32 位计算机按字节编址、小端、int 为 4B，按自然边界对齐。结构为 `struct record { int id; char name[10]; int salary; } employee[200];`，数组首址 A0B0H。若 employee[1].id 为 12345678H，其中 56H 的地址是多少？
5. 自拟：8 位补码最小负数取反加 1 得到自身，为什么不能据此说数学相反数等于自身？
6. 自拟：解释 8 位补码 -3 算术右移 1 位和 C 整数 $-3/2$ 为何可能不同。
7. 自拟：binary32 在 $[2^{25},2^{26})$ 的相邻间隔是多少？$2^{25}+1$ 按最近偶数舍入到哪里？

8. 自拟：列出恢复余数和不恢复余数法都必须满足的最终商余数关系。
9. 自拟：12345678H 采用大端和小端时，最低地址分别保存哪一字节？字节中的位需要反转吗?

2.11.3 参考答案

4. A0C5H。结构大小 20B，第二元素首址 A0C4H，小端 56H 位于其后 1B。
5. 同宽运算按模 256 保存位串；+128 不在 8 位补码范围，不能把位串绕回当作数学等式。
6. 算术右移复制符号位，-3 变 -2，相当于向负无穷取整；C 整数除法向零截断得 -1。还须核语言版本对负数右移的规定。
7. 间隔 4，$2^{25}+1$ 离 2^{25} 更近，舍入到 2^{25}。
8. 无符号被除数 $N=QD+R$，$D>0$ 时 $0\le R<D$；不恢复法可能需最终负余数校正。有符号还需给定商、余数符号及舍入约定。
9. 大端 12H、小端 78H；不反转字节内部位。

最小反例·成员后面仍可能补空隙

最小反例：int 之后的 char 只占 1B，但接下来的 short 起点仍需为 2 的倍数。因此 char 与 short 之间可能填充 1B。连续声明不代表在内存中毫无间隙；结构体末尾也可能有填充。